MyLocalAssistant

Tester Guide — Version 2.1 (Windows desktop, fully offline)

Internal — testers only

Document revision: 2.1 · April 2026

> **What you are testing.** MyLocalAssistant is a fully offline AI platform for small organizations. A local server hosts the LLM, agents, and a document-aware RAG pipeline; two desktop apps (Admin console and Chat Client) talk to it over the LAN. Everything runs on your PC — no data leaves it.

Table of contents

1. What's in the box
2. Where and how to download the installer
3. Installing the app
4. First run & the system tray launcher
5. Test accounts (usernames & passwords)
6. Using the Admin console
7. Using the Chat Client
8. Skills & plug-ins (owner-only)
9. Automatic updates
10. Uninstalling
11. Suggested test scenarios
12. Troubleshooting & logs
13. How to file feedback / bugs

1 · What's in the box

The single installer drops four executables that work together:

Executable	Role
`MyLocalAssistant.ServerHost.exe`	Tray-icon launcher. Starts/stops the server, monitors health, and pulls auto-updates from GitHub. **This is the only thing you need to launch.**
`MyLocalAssistant.Server.exe`	The ASP.NET Core service that runs the LLM, the RAG pipeline, authentication, and the REST + streaming API on `http://127.0.0.1:8080`. Spawned automatically by the tray app.
`MyLocalAssistant.Admin.exe`	WinForms console for system admins: users, departments, roles, models, RAG collections, audit, settings. Owner can also edit agents and global system prompt.
`MyLocalAssistant.Client.exe`	WinForms end-user chat app: log in, pick an agent, chat, attach a file to a turn, browse history.

System requirements

- Windows 10 21H2 or Windows 11 (x64).
- .NET 8 Desktop Runtime (the installer prompts you if it is missing).
- ≥ 8 GB RAM (16 GB recommended for larger models).
- ≥ 5 GB free disk for the app, plus space for any GGUF models you download (typically 2–8 GB each).
- Optional GPU acceleration: NVIDIA CUDA 12 or any Vulkan-capable GPU (LLamaSharp auto-selects the best back-end).

2 · Where and how to download the installer

1. Open `https://github.com/mguneskou/MyLocalAssistant/releases/latest` in your browser.
2. Under **Assets**, download `MyLocalAssistant-win-Setup.exe`.
3. If GitHub asks you to sign in (private repo while in beta), use the GitHub account that received the invite.

> **SmartScreen warning is expected.** The first beta builds are not yet code-signed, so Windows will show "*Windows protected your PC*". Click **More info** → **Run anyway**. This will go away once we attach an Authenticode certificate.

3 · Installing the app

1. Double-click `MyLocalAssistant-win-Setup.exe`.
2. The Velopack installer is silent — there is no wizard, no Next/Next dialog. After ~10–20 seconds you will see a tray icon appear.
3. The app installs **per-user** at `%LocalAppData%\MyLocalAssistant\current\` — no admin rights required.
4. A Start-menu shortcut named **MyLocalAssistant** is created automatically.

What gets created on disk

Path	Purpose
`%LocalAppData%\MyLocalAssistant\current\`	Program files (the four exes + native deps).
`%LocalAppData%\MyLocalAssistant\current\data\`	SQLite database `app.db` (users, conversations, audit).
`%LocalAppData%\MyLocalAssistant\current\models\`	GGUF model files you download from the Admin console.
`%LocalAppData%\MyLocalAssistant\current\vectors\`	LanceDB vector index for RAG.
`%LocalAppData%\MyLocalAssistant\current\logs\`	Server logs, rotated daily.
`%LocalAppData%\MyLocalAssistant\client.log`	Client-side crash log.

4 · First run & the system tray launcher

After install, the tray launcher is already running. Look for the **MyLocalAssistant** icon in the Windows notification area (you may need to click the `^` chevron).

Right-click the tray icon to see:

- **Status: Running / Stopped / Starting...** — live health-check (every 2 s) of the local server.
- **Open Admin console** — launches `MyLocalAssistant.Admin.exe`.
- **Open Chat Client** — launches `MyLocalAssistant.Client.exe`.
- **Open logs folder** · **Open install folder** — Explorer shortcuts.
- **Restart server** — kills + respawns the server child process.
- **Check for updates...** — manual update check (see §8).
- **About** — shows the installed version.
- **Quit** — stops the server and exits the launcher.

> **Tip.** Leave the launcher running. As long as the tray icon is there, the server is alive and the Admin / Client apps will connect instantly.

5 · Test accounts (usernames & passwords)

The server seeds two accounts on first launch. **Both passwords are intentionally weak — change them immediately if you ever expose the server beyond `127.0.0.1`.**

Tier	Username	Password	What they can do
Global admin (Owner)	`owner`	`owner`	Hidden top-tier account. Exclusive control over: • Which agents are *enabled* at all. • Each agent's **system prompt**. • The **global system prompt** applied to every conversation. Also has every system-admin permission. Does *not* appear in the Users tab and cannot be deleted from the UI. Rotated only by editing source & redeploying.
System admin	`admin`	`admin`	Bootstrap account. **Forced to change the password on first login.** Manages users, departments, roles, models, RAG collections, audit log, server settings. Cannot edit agents or the global system prompt — that's owner-only.
Test end-user(s)	*You create them.* Log in as `admin` → Users tab → New user.		Chat with whichever agents their **department** and **roles** grant access to. Cannot open the Admin console.

> **Security note.** The server binds to `127.0.0.1:8080` by default — only your PC can reach it. If you change `ServerSettings → BindAddress` to `0.0.0.0` to test multi-PC LAN scenarios, you *must* first rotate both passwords above and (ideally) put TLS in front.

6 · Using the Admin console

6.1 Sign in for the first time

1. Tray icon → **Open Admin console**.
2. The first time, the connection dialog appears. Server URL is pre-filled to `http://127.0.0.1:8080`. Click **Connect**.
3. Log in as `admin` / `admin`.
4. You will be forced to choose a new password (minimum 8 characters). Pick something memorable for testing — e.g. `Test1234!`.
5. Optionally, sign out and log back in as `owner` / `owner` to see the extra **Agents** tab and the **Global system prompt** editor (Settings tab, owner-only section).

6.2 The tabs

Tab	What to do here
Users	Create / edit / disable user accounts. Reset passwords. Assign roles + department.
Departments	Logical groups (e.g. *HR*, *Engineering*). Each user belongs to one. Used by RAG ACL.
Roles	Tags users can hold (multiple per user). Used to grant per-collection RAG read access.
Models	Browse the catalog, download a GGUF model, mark one as *active*. The active model is what every agent uses by default.
RAG Collections	Create a collection, ingest PDF / DOCX / XLSX / HTML / TXT / MD files, then expose it to a department + selected roles.
Agents (owner only)	Enable/disable agents, edit per-agent system prompts, pick a default model, bind which *skills* each agent is allowed to call.
Skills (owner only)	The skill catalog: built-in skills + installed plug-ins. Toggle Enabled, edit per-skill JSON config, hot-reload plug-ins. See §8.
Audit	Searchable log of every admin action and every chat turn (90-day retention by default).
Settings	Server bind address, JWT lifetimes, audit retention, LDAP config, ingestion limits, and (owner) the global system prompt.

6.3 The minimum setup to be able to chat

1. **Models** tab → pick a small model (e.g. *Phi-3-mini-4k-instruct Q4*, ~2.4 GB) → **Download** → **Set active**.
2. **Departments** tab → create "*Default*".
3. **Users** tab → create a user, e.g. `tester1`, password `Test1234!`, department *Default*, role *(none)*.
4. (Optional) **RAG Collections** → *New* → upload a PDF/DOCX → grant *Default* department read.
5. (Owner only — optional) **Agents** tab → make sure at least the *General* and *Documentation* agents are *Enabled*.

7 · Using the Chat Client

1. Tray → **Open Chat Client**.
2. First launch: confirm server URL `http://127.0.0.1:8080`, click **Connect**.
3. Log in as `tester1` (or whatever non-admin you created).
4. Left pane: list of agents you are allowed to use. Pick one.
5. Bottom pane: type your message, optionally attach a single file (📎) for that turn only, press `Ctrl` + `Enter` or click **Send**.
6. Replies stream in token-by-token. Use the toolbar to *Copy*, *Export*, or start a new conversation.
7. The left "History" pane keeps every conversation, grouped per agent.

> **Per-turn attachments are ephemeral.** They are parsed and inlined into that one prompt and then discarded — they do *not* become part of any RAG collection. To make a document permanently searchable, ingest it from the Admin console.

8 · Skills & plug-ins (owner-only)

Skills are tools an agent can call mid-conversation — e.g. evaluating a math expression, returning the current time, or searching a RAG collection for a citation. The catalog is **visible only to the global admin (`owner`)**; regular system admins cannot see the Skills tab and the underlying `/api/admin/skills` mutate endpoints reject their tokens.

8.1 Built-in skills (always present)

Skill id	What it does
`math.eval`	Evaluates an arithmetic expression (NCalc). No I/O, instant.
`time.now`	Returns the server's current UTC + local time.
`rag.search_collection`	Searches a named RAG collection and returns top-k passages with citations. Honors the user's department/role ACL — never leaks content the caller can't read.

8.2 How agents use skills

Tag-mode tool calling — model-agnostic, no JSON-grammar required:

1. The agent emits a block in its reply, e.g. `<tool>{"name":"math.eval","args":{"expr":"3*(4+5)"}}</tool>`.
2. The server intercepts the block, runs the skill (subject to the agent's capability binding), and feeds the result back into the same turn.
3. The model continues generating with the result in context, so the user sees one seamless reply.

8.3 The Skills tab

Log in as `owner` / `owner` → the **Skills** tab appears between Agents and Models.

- **Enabled** checkbox — disabled skills are hidden from every agent immediately.

- **Tools** button — inspect the JSON-schema of each tool the skill exposes.
- **Config...** button — edit the skill's per-instance JSON config (e.g. default collection name for `rag.search_collection`).
- **Source** column — *builtin* or *plugin*.
- **Signed by** column — the trusted-key id (from `config\trusted-keys\`) that signed the plug-in manifest. Empty for built-ins.
- **Reload plug-ins** toolbar button — rescans the server's `plugins\` folder, disposes old instances, registers fresh ones. Built-ins are untouched.

8.4 Binding skills to an agent

1. **Agents** tab (owner-only) → pick an agent.
2. Tick the skills you want this agent to be allowed to call. An agent with no skills bound is effectively chat-only.
3. Save. The change applies to the next chat turn — no restart required.

> **Why this matters during testing.** If you enable `rag.search_collection` on the *Documentation* agent, replies will start citing your RAG documents. If you remove it, the agent falls back to pure-LLM answers — useful for A/B testing whether RAG actually helps for a given query.

8.5 Installing a plug-in skill (out-of-process, sandboxed)

Plug-ins run as separate processes (JSON-RPC over stdio) inside a Windows **Job Object** with memory + CPU caps and no child-process / network rights by default. They must be signed.

1. Build or obtain a plug-in folder — the reference example is `plugins\echo\` in the repo.
2. Sign / pack it with the bundled CLI:

   ```
   SkillTools.exe pack --in plugins\echo --out my-skill.skill --key my-private.pem
   ```

3. Drop the *unzipped* plug-in folder into `%LocalAppData%\MyLocalAssistant\current\plugins\`.
4. Make sure the matching public key is in `%LocalAppData%\MyLocalAssistant\current\config\trusted-keys\`.
5. Skills tab → **Reload plug-ins**. The new skill appears with *Source* = *plugin* and *Signed by* = *<your key id>*.
6. Bind it to one or more agents (§8.4). Try it from the Chat Client.

> **Unsigned or untrusted plug-ins are silently skipped during reload.** If a plug-in doesn't show up after Reload, check the server log for "*plug-in rejected: signature invalid*" or "*unknown signing key*".

8.6 Tool-call statistics

The server keeps in-memory counters of how often each skill has been invoked since startup (`GET /api/admin/skills/stats`). A future Admin UI will surface them; for now you can hit the endpoint directly with the owner's JWT to validate that your test agent actually called the tool.

9 · Automatic updates

The tray launcher checks GitHub Releases for a newer version **every hour in the background**, plus once on startup (after a 5-second delay).

What happens when a new release is available

1. The launcher silently downloads only the delta package (usually a few MB).
2. On the next restart of the launcher, the update is applied and the new version starts.
3. To force this immediately: tray → **Check for updates...** → confirm the prompt → the launcher stops the server, applies the update, and restarts itself.

> **"Check for updates... → Auto-update only works on installed builds".** You will see this message if you ran the app from a development checkout (`dotnet run`) instead of via the Setup.exe. It is harmless — it just means there is nothing to update in place.

Where to see what version you are on

Tray → **About**. Compare against the latest tag on `https://github.com/mguneskou/MyLocalAssistant/releases`.

10 · Uninstalling

1. Right-click the tray icon → **Quit** (so the server releases its files).
2. Open Windows **Settings** → **Apps** → **Installed apps**, find **MyLocalAssistant**, click **Uninstall**.
3. This removes the program files at `%LocalAppData%\MyLocalAssistant\current\`.

Optional clean wipe (also removes your DB and downloaded models)

```
Remove-Item -Recurse -Force "$env:LocalAppData\MyLocalAssistant"
```

> Run the clean wipe *only* if you want a fresh tester install. It throws away every account you created, every conversation, every RAG collection, every downloaded model.

11 · Suggested test scenarios

11.1 Smoke test (5 min)

1. Install. Confirm tray icon shows *Status: Running* within 30 s.
2. Open Admin, log in as `admin` / `admin`, change password.
3. Download the smallest catalog model. Mark it active.
4. Create one user. Log into the Client as that user. Send "Hello, who are you?". Expect a streaming reply.

11.2 Full RAG round-trip (15 min)

1. Admin → RAG Collections → New collection "*Handbook*".
2. Ingest 2–3 PDF / DOCX files. Wait for the chunk count to settle.

3. Grant *Default* department *read*.
4. In the Client, select the *Documentation* agent. Ask a question whose answer is only in those docs.
5. Verify the reply *cites the source files* (filename + page) at the end.

11.3 ACL test (10 min)

1. Create two departments (*HR*, *Eng*) and two users (one per dept).
2. Create two collections, grant each to a different department.
3. Log in as the HR user; verify the Eng collection's content is **not** retrieved and the Eng-only agent (if any) does not appear in the agent list.

11.4 Owner overrides (10 min)

1. Log in as `owner` / `owner` in the Admin console.
2. Agents tab → disable the *General* agent. Verify it disappears from the Client's agent list within ~5 s after a refresh.
3. Edit a per-agent system prompt to "Always reply in Yoda voice." Send a chat turn → confirm the style change.
4. Settings → Global system prompt → set "Be concise, max 3 sentences." Verify every agent now obeys it.

11.5 Skills & plug-ins (10 min, owner-only)

1. Log in as `owner` / `owner`. Open the **Skills** tab — verify `math.eval`, `time.now`, `rag.search_collection` are listed and Enabled.
2. Agents tab → pick *General* → bind only `math.eval` + `time.now`. Save.
3. Chat as a regular user with *General*: "What is 17 * 23 + 5?" and "What time is it on the server?" — verify the answers are correct (the model invoked the tools).
4. Skills tab → disable `math.eval`. Re-ask the math question — verify the model now answers from its own (often wrong) arithmetic, proving the tool was actually doing the work.
5. (Optional) Drop the reference `plugins\echo\` folder into `%LocalAppData%\MyLocalAssistant\current\plugins\`, click **Reload plug-ins**, bind it to an agent, ask it to "echo hello world".

11.6 Auto-update flow (only if a new release has been published)

1. Note your current version (tray → About).
2. Tray → Check for updates… → confirm the upgrade prompt.
3. The launcher restarts. Reopen About → verify the version number changed.
4. Confirm your DB / users / RAG collections survived the upgrade.

11.7 Resilience test (5 min)

1. Tray → Restart server. Verify the Admin and Client apps reconnect within ~10 s.
2. Force-kill `MyLocalAssistant.Server.exe` from Task Manager. The launcher should restart it within a few seconds (status flips Stopped → Starting → Running).

12 · Troubleshooting & logs

Logs to grab when reporting a bug

What broke	Log file
Server / API / RAG	`%LocalAppData%\MyLocalAssistant\current\logs\server-*.log` (latest)
Client crash on startup	`%LocalAppData%\MyLocalAssistant\client.log`
Tray launcher / updater	Tray → **Open logs folder**

Common issues

Symptom	Likely cause & fix
Tray status is permanently *Stopped*.	Port `8080` is in use. Quit the launcher, change `ServerSettings.json` → `BindPort`, restart.
"Cannot connect to server" in Admin/Client.	Tray icon present? If yes, wait 10 s — the server is still loading the model. If no, run the launcher from the Start menu.
Model download stalls.	Some catalog mirrors throttle. Cancel and retry, or pick a different model.
Chat reply says "no model loaded".	Admin → Models → mark a downloaded model as **Active**.
SmartScreen blocks the installer.	More info → Run anyway. Builds are unsigned during beta.
Forgot the new `admin` password.	Log in as `owner` / `owner`, Users tab → reset password on the admin row.
Both passwords forgotten.	Quit, delete `%LocalAppData%\MyLocalAssistant\current\data\app.db`, restart the launcher. The seed accounts come back. *This wipes all users and history.*

13 · How to file feedback / bugs

Open an issue at `https://github.com/mguneskou/MyLocalAssistant/issues` and please include:

- App version (tray → About).
- Windows build (`winver`).
- What you did, what you expected, what happened.
- The relevant log file from §12 (zip and attach).

- A screenshot if it's a UI issue.

© 2026 MyLocalAssistant — Tester Guide v2.1. Distributed to invited testers only. Default credentials in this document apply only to fresh installs and must be rotated before any non-loopback deployment.